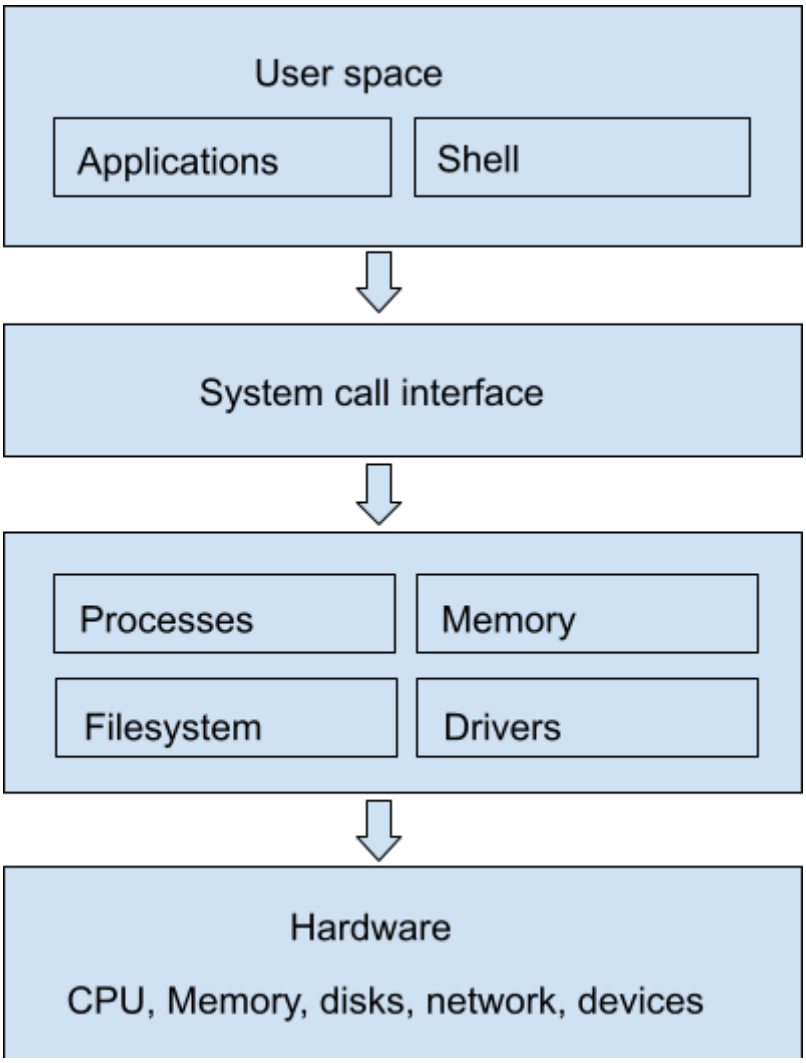
CAREERBYTECODE INTERNSHIP

WEEK 1 ASSIGNMENT

1. **Explain the architecture of Linux with a neat diagram and describe the role of the kernel, shell, space, and hardware.**



ROLES:

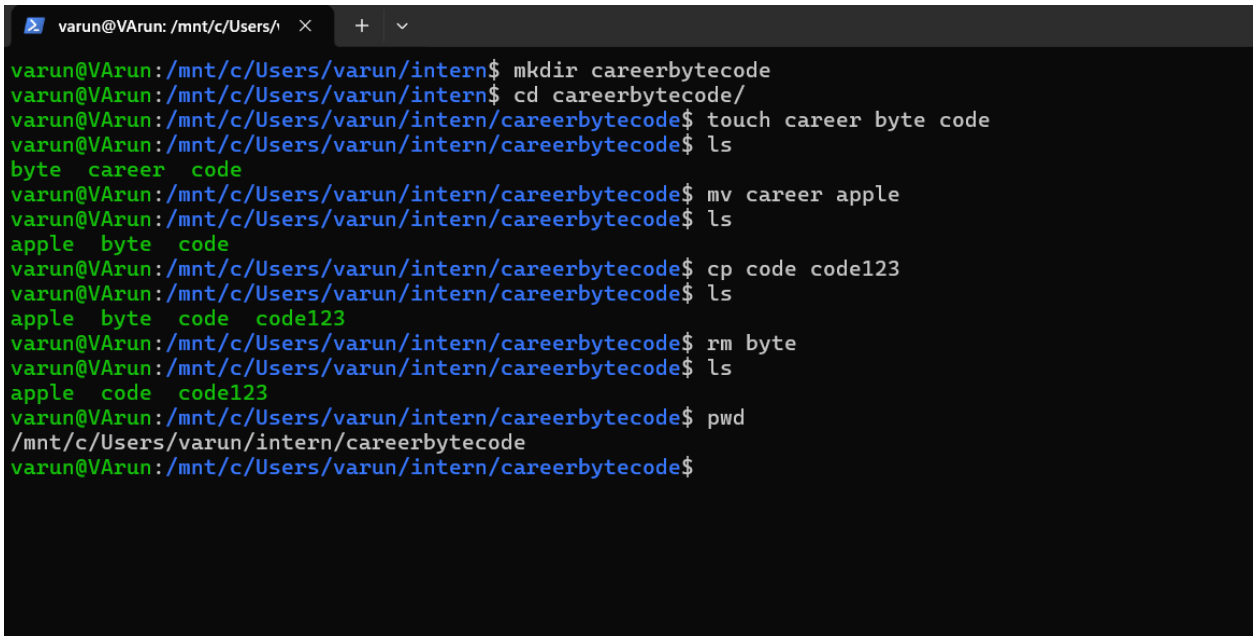
1. **HARDWARE**
    a. Hardware is the physical foundation - CPU, RAM, storage disks, network cards, keyboard and other peripherals.
    b. Hardware can't be used directly by programs, every access must go through the kernel.
    c. This isolation is what makes the system stable and secure
2. **Kernal**
    a. Kernal is the core of theOS, loaded into memory at boot and running in privileged mode with full hardware access.
    b. It's main jobs are, process management, memory management, etc.. and it translating the requests like "write this block" into hardware specific commands.
    c. It also handles networking and security permissions.
3. **System call interface**
    a. System call interface is the gate between user space and the kernel.
    b. When a program needs anything privileged - open a file, spawn a process, send a packet - it invokes syscall like open(), fork(), read(), write().
4. **Shell**
    a. Shell is a user-space program that acts a command interpreter.
    b. It reads what you type like "ls -ltr, cat, etc…. " , parses it, it locates the binary, and asks the kernel to execute it via syscalls (fork() + exec()), then displays the output.
    c. Examples of shells are Bash, Zsh, Fish shell, etc…
5. **User space**
    a. User space is everything that isn't the kernel: applications,daemons, container runtimes and the shell itself.
    b. Process here run in unprivileged mode with isolated virtual memory, so one crashing app can't corrupt the kernal or other processes

2. Differentiate between Linux and Unix by listing at least eight differences.

Feature	**Linux**	**Unix**
Origin	1991 (Linus Torvalds/community)	1969 (AT&T Bell Labs)
Licensing	Opensource (GPL - General public license)	Proprietary (Various licenses)
Development	Community-driven global development	Vendor-driven(IBM, Oracle, HP)
Default Shell	Bash (Bourne Again Shell)	Bourne Shell (sh) or Korn shell (ksh)
Kernel Type	Monolithic (Modular)	Monolithic (Traditional)
GUI	Diverse (GNOME, KDE, XFCE)	Limited (Common Desktop Environment)
File systems	Ext4, XFS, Btrfs, ZFS	UFS, JFS, ZFS, HFS
Hardware	Anything from a watch to a supercomputer	Specific high-end servers/workstations

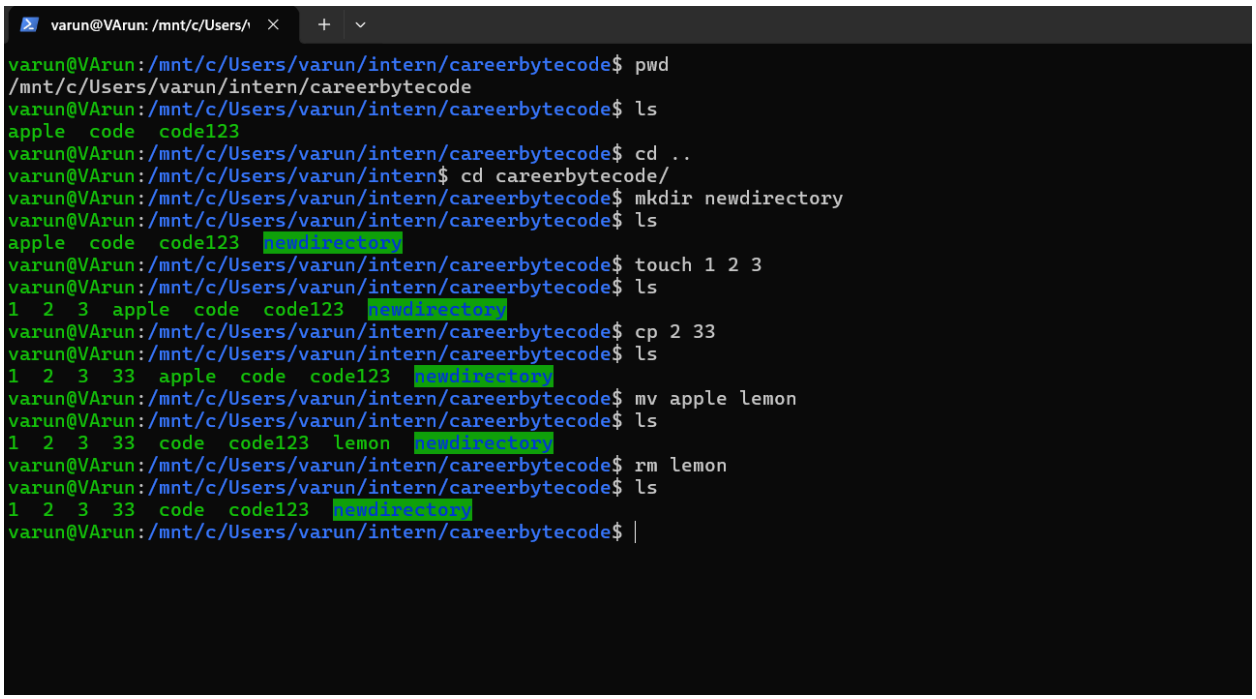
3. Write the Linux commands to perform the following tasks

a. **Create a directory named careerbytecode**
b. **Create three files inside it**
c. **Rename one file**
d. **Copy one file**
e. **Delete one file**
f. **Display the current working directory**

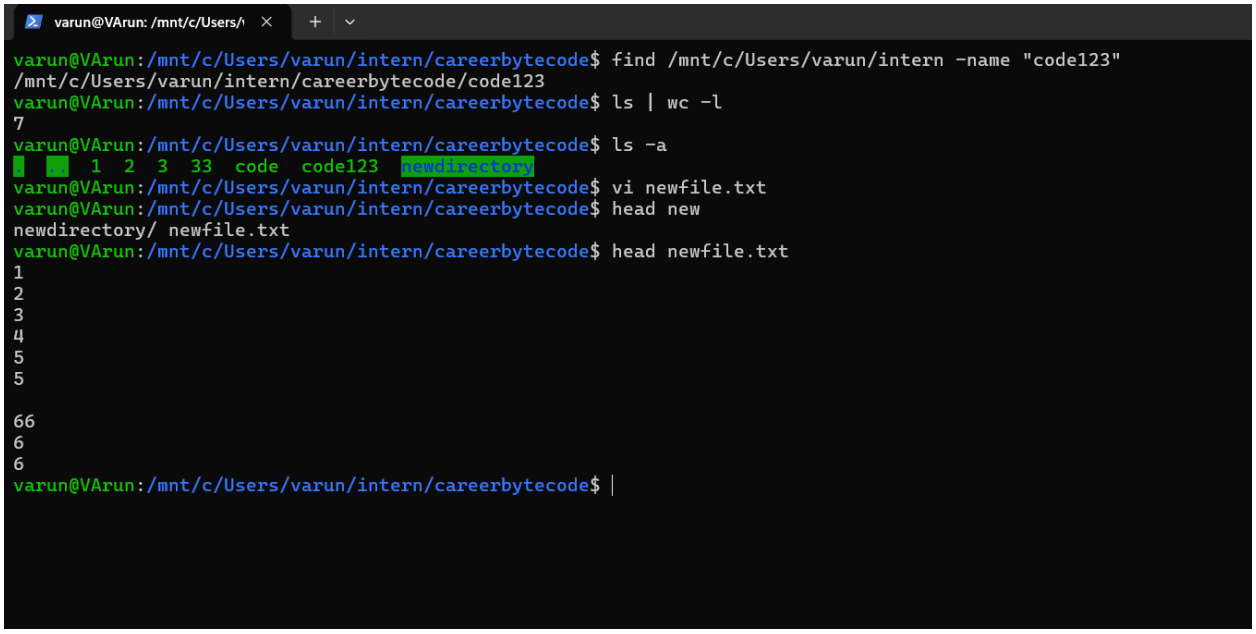


4. Explain the purpose of the following commands with one example each: pwd, ls, cd, mkdir, touch , cp, mv and rm.

pwd = It shows the present working directory
ls = It list the files files and directories of working repository
cd = change directory, it helps to move from one directory to another directory
mkdir = It creates the new directory
touch = It creates new empty file
cp = It helps to copy file or directory from one destination to another destination
mv = It helps to move from one destination to another destination and also helps to rename the files, directory.
rm = It's used to remove files



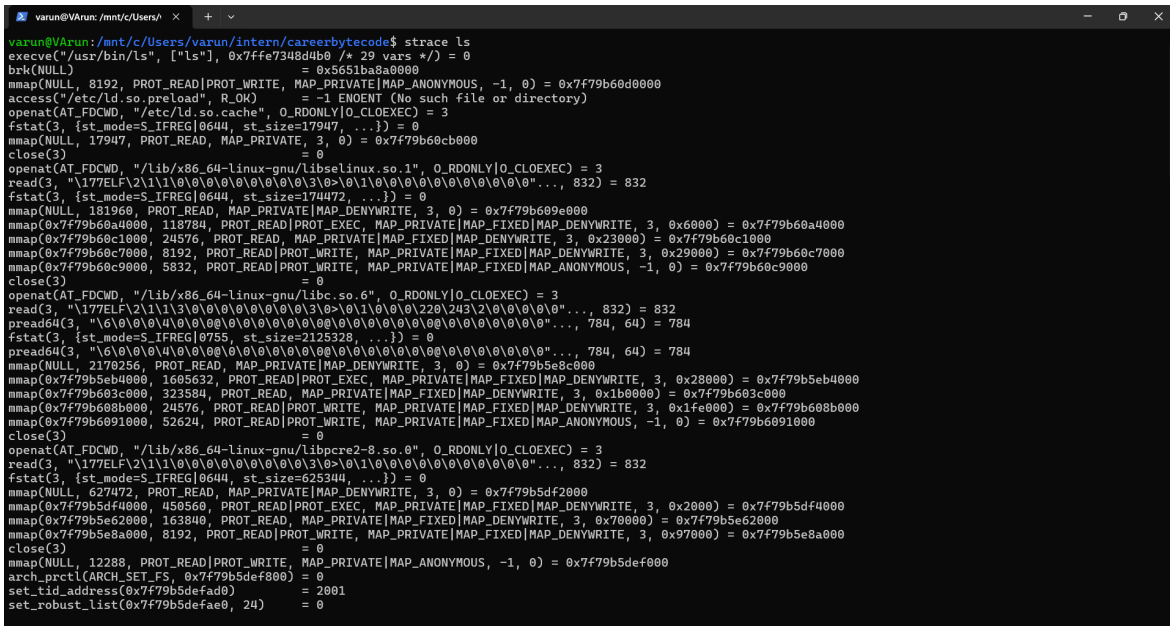
5. Write the commands to search for a file, count the number of files in directory, display hidden files, and display the first 10 lines of a file



6. Research and explain five Linux commands that were not covered during the training. Include one example for each.

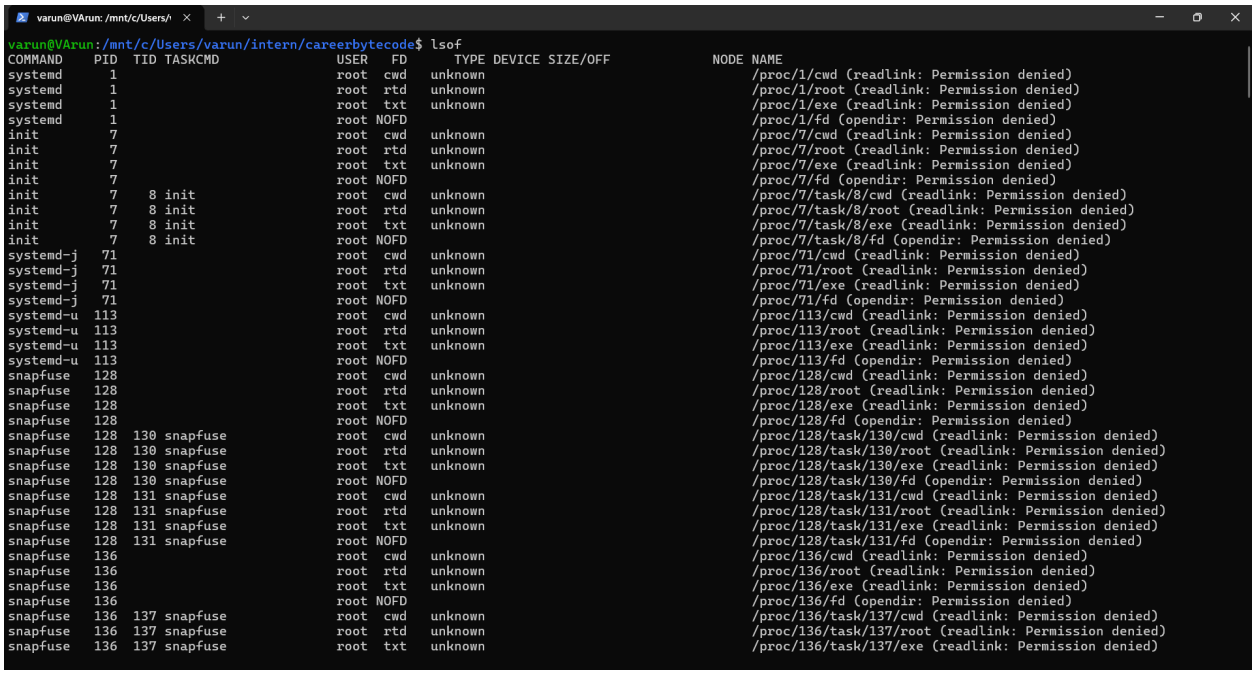
‘Strace’ command:

Strace command traces system calls and signals made by a process. It lets you see what a program is actually doing , without needing its source code



lsof - List Open Files

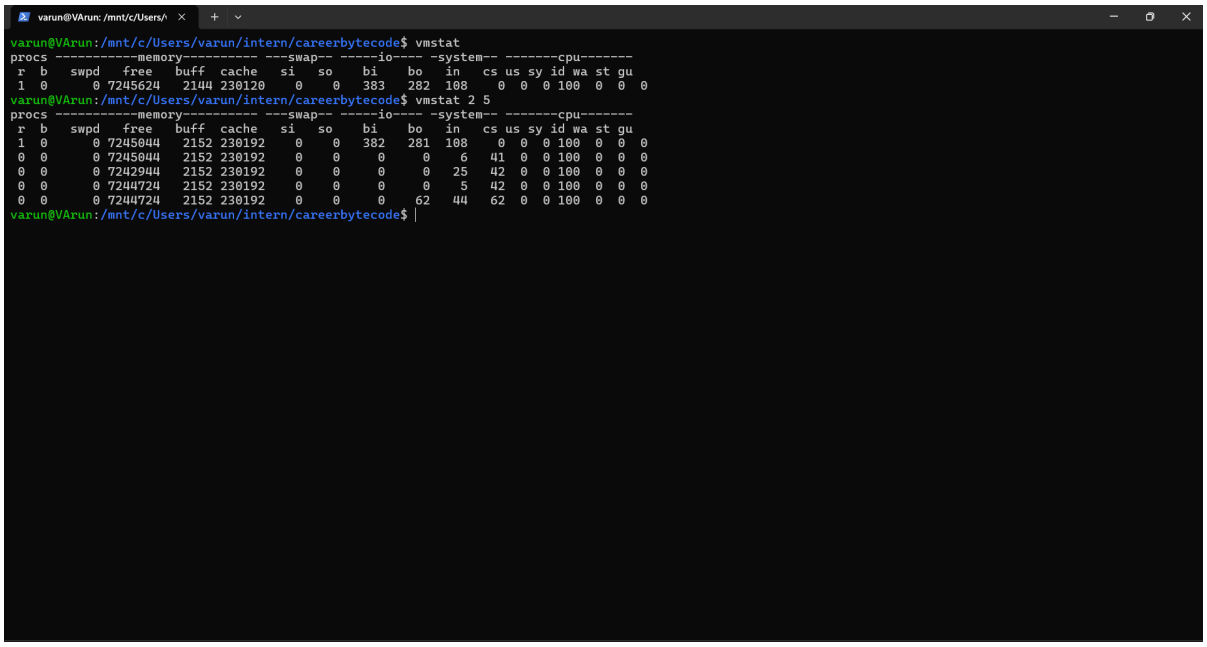
Shows every file, socket, port a process has open, In Linux “everything is a file”, so this covers nearly everything



```
varun@VArun:/mnt/c/Users/varun/intern/careerbytecode$ lsof
COMMAND    PID  TID TASKCMD              USER   FD      TYPE DEVICE SIZE/OFF            NODE NAME
systemd      1                          root  cwd   unknown                                 /proc/1/cwd (readlink: Permission denied)
systemd      1                          root  rtd   unknown                                 /proc/1/root (readlink: Permission denied)
systemd      1                          root  txt   unknown                                 /proc/1/exe (readlink: Permission denied)
systemd      1                          root NOFD                                           /proc/1/fd (opendir: Permission denied)
init         7                          root  cwd   unknown                                 /proc/7/cwd (readlink: Permission denied)
init         7                          root  rtd   unknown                                 /proc/7/root (readlink: Permission denied)
init         7                          root  txt   unknown                                 /proc/7/exe (readlink: Permission denied)
init         7                          root NOFD                                           /proc/7/fd (opendir: Permission denied)
init         7    8 init                root  cwd   unknown                                 /proc/7/task/8/cwd (readlink: Permission denied)
init         7    8 init                root  rtd   unknown                                 /proc/7/task/8/root (readlink: Permission denied)
init         7    8 init                root  txt   unknown                                 /proc/7/task/8/exe (readlink: Permission denied)
init         7    8 init                root NOFD                                           /proc/7/task/8/fd (opendir: Permission denied)
systemd-j   71                          root  cwd   unknown                                 /proc/71/cwd (readlink: Permission denied)
systemd-j   71                          root  rtd   unknown                                 /proc/71/root (readlink: Permission denied)
systemd-j   71                          root  txt   unknown                                 /proc/71/exe (readlink: Permission denied)
systemd-j   71                          root NOFD                                           /proc/71/fd (opendir: Permission denied)
systemd-u  113                          root  cwd   unknown                                 /proc/113/cwd (readlink: Permission denied)
systemd-u  113                          root  rtd   unknown                                 /proc/113/root (readlink: Permission denied)
systemd-u  113                          root  txt   unknown                                 /proc/113/exe (readlink: Permission denied)
systemd-u  113                          root NOFD                                           /proc/113/fd (opendir: Permission denied)
snapfuse   128                          root  cwd   unknown                                 /proc/128/cwd (readlink: Permission denied)
snapfuse   128                          root  rtd   unknown                                 /proc/128/root (readlink: Permission denied)
snapfuse   128                          root  txt   unknown                                 /proc/128/exe (readlink: Permission denied)
snapfuse   128                          root NOFD                                           /proc/128/fd (opendir: Permission denied)
snapfuse   128  130 snapfuse            root  cwd   unknown                                 /proc/128/task/130/cwd (readlink: Permission denied)
snapfuse   128  130 snapfuse            root  rtd   unknown                                 /proc/128/task/130/root (readlink: Permission denied)
snapfuse   128  130 snapfuse            root  txt   unknown                                 /proc/128/task/130/exe (readlink: Permission denied)
snapfuse   128  130 snapfuse            root NOFD                                           /proc/128/task/130/fd (opendir: Permission denied)
snapfuse   128  131 snapfuse            root  cwd   unknown                                 /proc/128/task/131/cwd (readlink: Permission denied)
snapfuse   128  131 snapfuse            root  rtd   unknown                                 /proc/128/task/131/root (readlink: Permission denied)
snapfuse   128  131 snapfuse            root  txt   unknown                                 /proc/128/task/131/exe (readlink: Permission denied)
snapfuse   128  131 snapfuse            root NOFD                                           /proc/128/task/131/fd (opendir: Permission denied)
snapfuse   136                          root  cwd   unknown                                 /proc/136/cwd (readlink: Permission denied)
snapfuse   136                          root  rtd   unknown                                 /proc/136/root (readlink: Permission denied)
snapfuse   136                          root  txt   unknown                                 /proc/136/exe (readlink: Permission denied)
snapfuse   136                          root NOFD                                           /proc/136/fd (opendir: Permission denied)
snapfuse   136  137 snapfuse            root  cwd   unknown                                 /proc/136/task/137/cwd (readlink: Permission denied)
snapfuse   136  137 snapfuse            root  rtd   unknown                                 /proc/136/task/137/root (readlink: Permission denied)
snapfuse   136  137 snapfuse            root  txt   unknown                                 /proc/136/task/137/exe (readlink: Permission denied)
```

vmstat command:

vmstat is Virtual memory statistics command is a built-in performance monitoring tool that reports real-time statistics about system processes , physical and virtual memory, paging, block I/O, and cpu activity



```
varun@VArun:/mnt/c/Users/varun/intern/careerbytecode$ vmstat
procs -----------memory---------- ---swap-- -----io---- -system-- -------cpu-------
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st gu
 1  0      0 7245624   2144 230120    0    0   383   282  108    0  0  0 100  0  0  0
varun@VArun:/mnt/c/Users/varun/intern/careerbytecode$ vmstat 2 5
procs -----------memory---------- ---swap-- -----io---- -system-- -------cpu-------
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st gu
 1  0      0 7245044   2152 230192    0    0   382   281  108    0  0  0 100  0  0  0
 0  0      0 7245044   2152 230192    0    0     0     0    6   41  0  0 100  0  0  0
 0  0      0 7242944   2152 230192    0    0     0     0   25   42  0  0 100  0  0  0
 0  0      0 7244724   2152 230192    0    0     0     0    5   42  0  0 100  0  0  0
 0  0      0 7244724   2152 230192    0    0     0    62   44   62  0  0 100  0  0  0
varun@VArun:/mnt/c/Users/varun/intern/careerbytecode$
```

watch command:

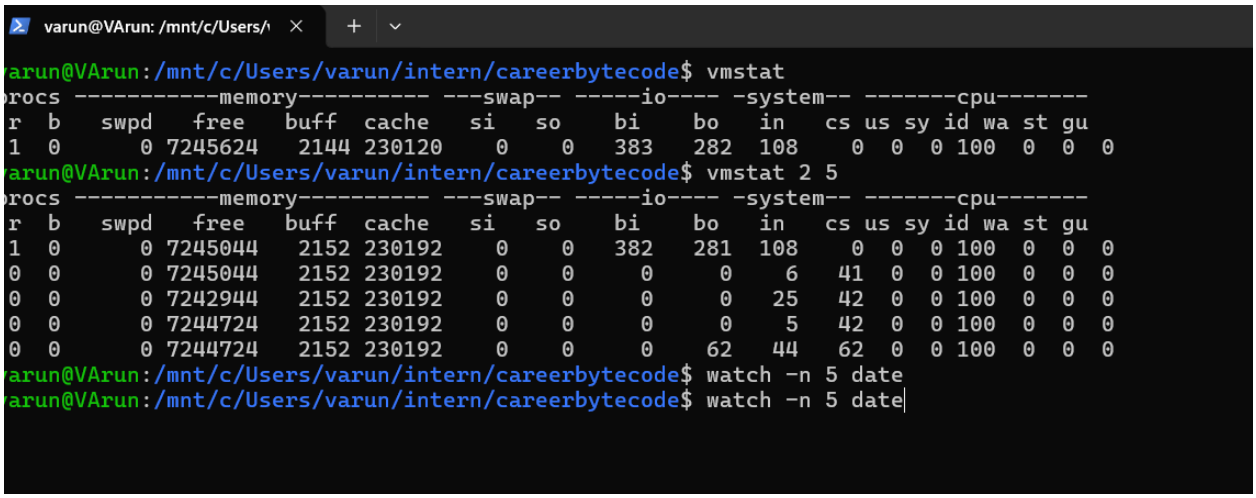
watch comand is command line utility used to execute a program repeatedly at a regular intervals, displaying the output in realtime.

watch [OPTIONS] command

Flag :

-n = seconds

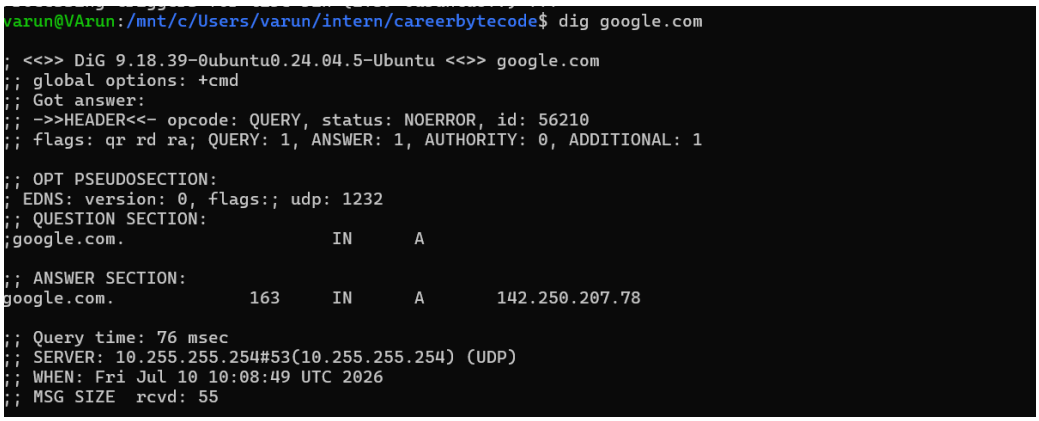
-d = difference between updates





dig command:

Domain Information Groper (dns lookup tool) is used for querying Domain Name System (DNS) name servers.



7. Explain the purpose of the following Linux directories: /home, /etc/ , /var, /usr, /opt, /tmp, /boot, and /dev.

/home → Home directory, where regular users store their data and configuration files.
/etc → system specific configuration files
/var → files that dynamically change, such as databases, cache directories, log files, Website content, etc.. might be found under /var.
/usr → installed software, shared libraries, including files, and read-only program data. Subdirectories include : /usr/bin , user commands, /usr/sbin: system administration commands, /usr/local: locally customized software
/opt → standard system directory reserved for storing optional, standalone and third party software packages that are not managed by the core operating system's package manager.
/tmp → a world-writable space for temporary files.
/boot → files to start the boot process.
/dev → special device files that the system uses to access hardware.

8. Differentiate between numeric and symbolic file permissions with suitable examples using chmod.
Three permissions = read, write, execute, Three groups = owner, group, others

Permission	**Value**
Read (r)	4
Write (w)	2
Execute (x)	1
none	0

Octal	**Binary**	**Meaning**
7	rwx	Read + write + execute
6	rw-	Read + write
5	r-x	Read + execute
4	r - -	Read only
0	- - -	No access

9. Your Linux server is running slowly. List the commands you would use to identify theproblem and explain why you selected them.

Step 1 : check the load average of the system

Uptime , nproc

Shows the load average (1,5,15 mins) and then compare with the nproc command (it shows the cpu count). If the load is higher than your core count, the server is overloaded.

```
root@ubuntu:~$ uptime
 16:04:27 up 15 min,  0 user,  load average: 0.01, 0.02, 0.00
root@ubuntu:~$
```

Step 2 : check what's using the cpu and memory

Top, htop

To check which processes at top taking the highest %cpu , %memory

```
top - 16:04:59 up 16 min,  0 user,  load average: 0.01, 0.01, 0.00
Tasks: 125 total,   1 running, 124 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   1903.2 total,    873.5 free,    416.1 used,    781.5 buff/cache
MiB Swap:   1024.0 total,   1024.0 free,      0.0 used.   1487.1 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0   22096  13276   9504 S   0.0   0.7   0:01.34 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_g
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_p
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
      8 root      20   0       0      0      0 I   0.0   0.0   0:00.03 kworker/0:0-inet_frag_wq
     12 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_pe
     13 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_kthread
     14 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
     15 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_trace_kthread
     16 root      20   0       0      0      0 S   0.0   0.0   0:00.04 ksoftirqd/0
     17 root      20   0       0      0      0 I   0.0   0.0   0:00.06 rcu_preempt
     18 root      rt   0       0      0      0 S   0.0   0.0   0:00.00 migration/0
     19 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/0
     20 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
     21 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kdevtmpfs
     22 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-inet_
     23 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kauditd
     24 root      20   0       0      0      0 S   0.0   0.0   0:00.00 khungtaskd
     26 root      20   0       0      0      0 S   0.0   0.0   0:00.00 oom_reaper
     27 root      20   0       0      0      0 I   0.0   0.0   0:00.02 kworker/u2:2-events_power_efficient
     28 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-write
     29 root      20   0       0      0      0 S   0.0   0.0   0:00.02 kcompactd0
     30 root      25   5       0      0      0 S   0.0   0.0   0:00.00 ksmd
     31 root      39  19       0      0      0 S   0.0   0.0   0:00.00 khugepaged
     32 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-kinte
     33 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-kbloc
     34 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-blkcg
     35 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 irq/9-acpi
```

Step 3 : check memory usage

Free -h

To check the memory usage, if its very low, you're out of ram

```
root@ubuntu:~$ free -h
               total        used        free      shared  buff/cache   available
Mem:           1.9Gi       416Mi       873Mi       1.1Mi       781Mi       1.5Gi
Swap:          1.0Gi          0B       1.0Gi
root@ubuntu:~$
```

Step 4 : check if it's swapping

Vmstat

Vmstat 3 5

Updates every 3 secs , stops automatically after 5 reports

We have to look at the columns like si (amount of memory swapped in from disk per second) and so (amount of memory swapped out to disk per second) , if they keep showing numbers above 0, your server is swapping (using disk as RAM) , it will make make your server very slow.

```
root@ubuntu:~$ vmstat
procs -----------memory---------- ---swap-- -----io---- -system-- -------cpu-------
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st gu
 1  0      0 894452  41356 758880    0    0   661   367  125    3  1  1 98  0  0  0
root@ubuntu:~$ vmstat 3 5
procs -----------memory---------- ---swap-- -----io---- -system-- -------cpu-------
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st gu
 2  0      0 901456  41372 758892    0    0   598   332  121    2  1  1 98  0  0  0
 0  0      0 901456  41372 758892    0    0     0     0   78  231  0  0 100  0  0  0
 0  0      0 901456  41372 758892    0    0     0     0   71  228  0  0 100  0  0  0
 0  0      0 901456  41372 758892    0    0     0     0   67  208  0  0 100  0  0  0
 0  0      0 901456  41372 758892    0    0     0     1   71  227  0  0 100  0  0  0
root@ubuntu:~$
```

Step 5 : check the disk usage

df -h

If any disks shows 100% or close to that, it will also make the server slow

```
root@ubuntu:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           191M  996K  190M   1% /run
/dev/vda1        19G   19G  2.1M 100% /
tmpfs           952M   84K  952M   1% /dev/shm
tmpfs           5.0M     0  5.0M   0% /run/lock
/dev/vda16      881M  117M  703M  15% /boot
/dev/vda15      105M  6.2M   99M   6% /boot/efi
root@ubuntu:~$
```

Step 6 : Check the disk speed

Iostat

Iostat -xz 1

x = extended metrics, z = omit the idle

Look at the %util. If a disk is near 100%, the disk is the bottleneck



```
root@ubuntu:~$ iostat -xz 1
Linux 6.8.0-124-generic (ubuntu)        07/10/26        _x86_64_        (1 CPU)

avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           0.65    0.34    0.47    0.28    0.18   98.08

Device            r/s     rkB/s   rrqm/s  %rrqm r_await rareq-sz     w/s     wkB/s   wrqm/s  %wrqm w_await wareq-sz     d/s     dkB/s   drqm/s  %
drqm d_await dareq-sz     f/s f_await  aqu-sz  %util
vda             12.00    523.15     3.73  23.69    0.58    43.61    2.56    291.02    19.23  88.26   10.10   113.79    2.06  11591.85     0.00
0.00    0.17  5615.42    0.17    9.78    0.03   0.52


avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           0.98    0.00    0.98    0.00    0.00   98.04

Device            r/s     rkB/s   rrqm/s  %rrqm r_await rareq-sz     w/s     wkB/s   wrqm/s  %wrqm w_await wareq-sz     d/s     dkB/s   drqm/s  %
drqm d_await dareq-sz     f/s f_await  aqu-sz  %util


avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           0.00    0.00    0.00    0.00    0.00  100.00

Device            r/s     rkB/s   rrqm/s  %rrqm r_await rareq-sz     w/s     wkB/s   wrqm/s  %wrqm w_await wareq-sz     d/s     dkB/s   drqm/s  %
drqm d_await dareq-sz     f/s f_await  aqu-sz  %util


avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           0.00    0.00    0.00    0.00    0.00  100.00

Device            r/s     rkB/s   rrqm/s  %rrqm r_await rareq-sz     w/s     wkB/s   wrqm/s  %wrqm w_await wareq-sz     d/s     dkB/s   drqm/s  %
drqm d_await dareq-sz     f/s f_await  aqu-sz  %util
vda             0.00     0.00     0.00   0.00    0.00     0.00    1.00     4.00     0.00   0.00    1.00     4.00    0.00     0.00     0.00
0.00    0.00     0.00    0.00    0.00    0.00   0.00
```

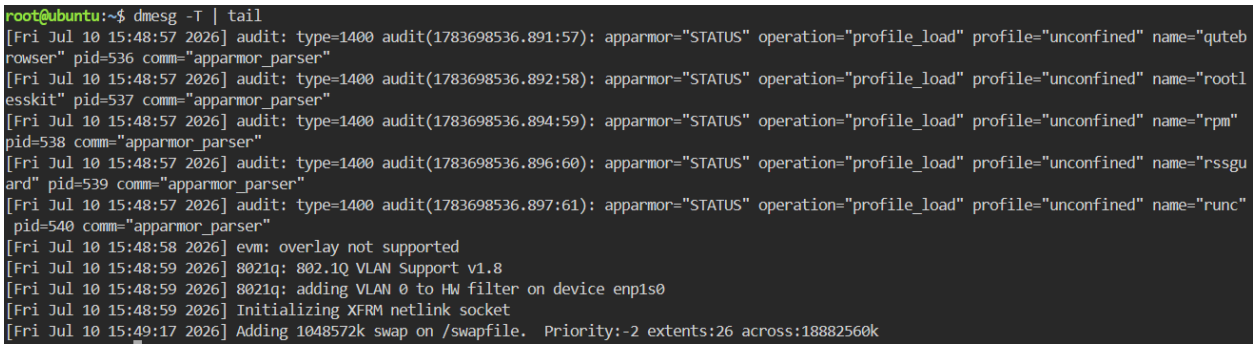
Step 7 : Read the logs to find the cause

Dmesg

Dmesg -T | tail

T = human readable time, tail = final 10 lines

This shows the hardware error, or if the system killed a process for too much memory OOM killer (out-of-memory killer).



```
root@ubuntu:~$ dmesg -T | tail
[Fri Jul 10 15:48:57 2026] audit: type=1400 audit(1783698536.891:57): apparmor="STATUS" operation="profile_load" profile="unconfined" name="quteb
rowser" pid=536 comm="apparmor_parser"
[Fri Jul 10 15:48:57 2026] audit: type=1400 audit(1783698536.892:58): apparmor="STATUS" operation="profile_load" profile="unconfined" name="rootl
esskit" pid=537 comm="apparmor_parser"
[Fri Jul 10 15:48:57 2026] audit: type=1400 audit(1783698536.894:59): apparmor="STATUS" operation="profile_load" profile="unconfined" name="rpm"
pid=538 comm="apparmor_parser"
[Fri Jul 10 15:48:57 2026] audit: type=1400 audit(1783698536.896:60): apparmor="STATUS" operation="profile_load" profile="unconfined" name="rssgu
ard" pid=539 comm="apparmor_parser"
[Fri Jul 10 15:48:57 2026] audit: type=1400 audit(1783698536.897:61): apparmor="STATUS" operation="profile_load" profile="unconfined" name="runc"
 pid=540 comm="apparmor_parser"
[Fri Jul 10 15:48:58 2026] evm: overlay not supported
[Fri Jul 10 15:48:59 2026] 8021q: 802.1Q VLAN Support v1.8
[Fri Jul 10 15:48:59 2026] 8021q: adding VLAN 0 to HW filter on device enp1s0
[Fri Jul 10 15:48:59 2026] Initializing XFRM netlink socket
[Fri Jul 10 15:49:17 2026] Adding 1048572k swap on /swapfile.  Priority:-2 extents:26 across:18882560k
```

This is the flow troubleshooting process of a linux server, why its works slow, what happen, how did happen , why its happen , those answers are we can find with using these troubleshooting methods.

10. A user cannot access a file stored in the server. Describe the troubleshooting steps you would follow.

Step 1: Get the exact error message

Ask the user, what error message you are facing, like permission denied, the file not found, timeout, disk full, these are the words where you have to look for troubleshooting.

Step 2: Check the file exists

Ls -l /path/file

If it's not there, the file may be moved or deleted or the path is wrong. Then fix it

Step 3: check permissions

Ask the user have rights to open the file?

If he is not having permissions to read, write or execute. Give permissions based on needs .

Step 4: check ACL / SELinux

Even now, the user couldn't access the files, check the access control list permissions and selinux permissions. Sometimes it can block access even when normal permissions looks correct

Step 5: Check the disk / filesystem

Sometimes a full disk or a readonly filesystem both shows "can't access"

Step 6: check the network

Only if the file is on the remote server like nfs, smb, etc.., is the drive actually mounted?

Mount

Ping servername

Step 7: check for locks

Like another process holding the file.

Lsof /path/file

Step 8: final step, verify as a user

Apply the smallest fix for the cause, then verify as a user

Sudo -u username cat /path/file

11. Explain the complete Git workflow from creating a repository to pushing code to github?

Step1: set the git identity (onetime per machine)

Gti config - - global user.name "arunprakash"

Git config - - global user.email "prakasarun484@gmail.com"

Step 2: create the local repo

Mkdir week-1-careerbytecode-assesment

Cd week-1-careerbytecode-assesment

Git init → initialize the project

Step 3: check status

Git status

Step 4: stage your changes

Git add files

Git add . #add everything in the repo

Step 5: git commit

Git commit -m "add the project"

Step 6: create the github repository

Create a empty repo, using https protocol and the authorize via personal access token in github

Step 7: connect to local repo to github "via remote"

Git remote add origin "git@github.com/user/repo.git"

Git remote -v

Step 8: push to github from local repo

Git branch -M main

Git push -u origin main

12. Write the git commands required to initialize a repository, add files, commit changes, create a branch, merge it, and push the code.

description	**Git commands**
Initialize the repo	Git init
Add files	Git add .
Commit changes	Git commit -m "commit messages"
Create a branch	Git branch -M main
Merge it	Git merge <feature branch>
Push the code	Git push -u origin main

13. Research any three git commands that were not discussed during the training and explain their purpose

1. **Git stash**
    a. It takes your uncommitted messages saves them somewhere safe, and gives back a clean working directory - without making a commit
    b. " git stash (stash your changes, working directory goes clean ", git stash pop (bring the changes back and remove them from stash " , etc..
2. **Git cherry-pick**
    a. It lets you grab a specific commit from somewhere else and apply it onto your current branch - just that one commit, not everything around it. It's like plucking a single commit and copying it over.
    b. git cherry-pick <commit-id> , …
3. **Git diff**
    a. Outputs the differences between two things, such as commits and branches
    b. git diff <commit-id-1> <commit-id-2>

14. What is github? Explain repository, clone, fork, pull requests, and issues

Github:

It's a cloud based platform, where developer store their code, manages, track and collaborate on code.

It allows developers to work together on projects and it supports both private and public repositories.

Repository:

A github repository is where your project files and version history are stored. You have public and private repositories depending on the project needs.

Clone:

Cloning is downloading a complete copy of remote repository onto your machine , all the codes, entire history of files and branches

Git clone <github-url>

Fork:

A fork is your own personal copy of someone else's repository, living on your github account.

Pull requests:

Pul requests is a proposal to merge your changes into another branch or repo. It's wrapped in review and discussion interface.

Issues:

Issues are github's built-in tracker for tasks , bugs, feature requests, and discussions.

15. Explain how GitHub helps multiple developers work on the same project without losing each other's changes.

1. Everyone get their full copy of the project. Nobody edits one shared live file, so there's no way to accidentally erase each other.
2. Each person works on their own branch, an isolated copy where they can make changes without touching others work
3. When the work is ready, git merges the branches together. It combines, instead of overwriting the files
4. Before push, you pull first to grab everyone's latest changes. And if you try to push it , git rejects it and makes you pull first.

16. Write a bash shell script that accepts a user's name, age, and department as input and displays the entered information.

```
#!/bin/bash

# Prompt for user input
read -p "Enter your name: " name
read -p "Enter your age: " age
read -p "Enter your department: " department

# Display the entered information
echo ""
echo "----- Information -----"
echo "Name:       $name"
echo "Age:        $age"
echo "Department: $department"
echo "-----------------------"
```

```
varun@VArun:/mnt/c/Users/varun/intern/careerbytecode$ vi new.sh
varun@VArun:/mnt/c/Users/varun/intern/careerbytecode$ bash new.sh
Enter your name: arun
Enter your age: 24
Enter your department: civil

----- Information -----
Name:       arun
Age:        24
Department: civil
-----------------------
```

17. Explain the purpose of the following bash special variables with examples: $0, $1, $2, $#, $?, $@, and $* .

Special variables	description
$0	The name of the script or shell itself
$1	Positional parameters of first arguments passed to the script
$2	Positional parameters of second argument passed to the script
$#	The count of arguments passed
$?	Exit status, 0 means success, non-zero means failure or error code
$@	It keeps each argument exactly as it was passed.
$*	All arguments into a single string

```
#!/bin/bash

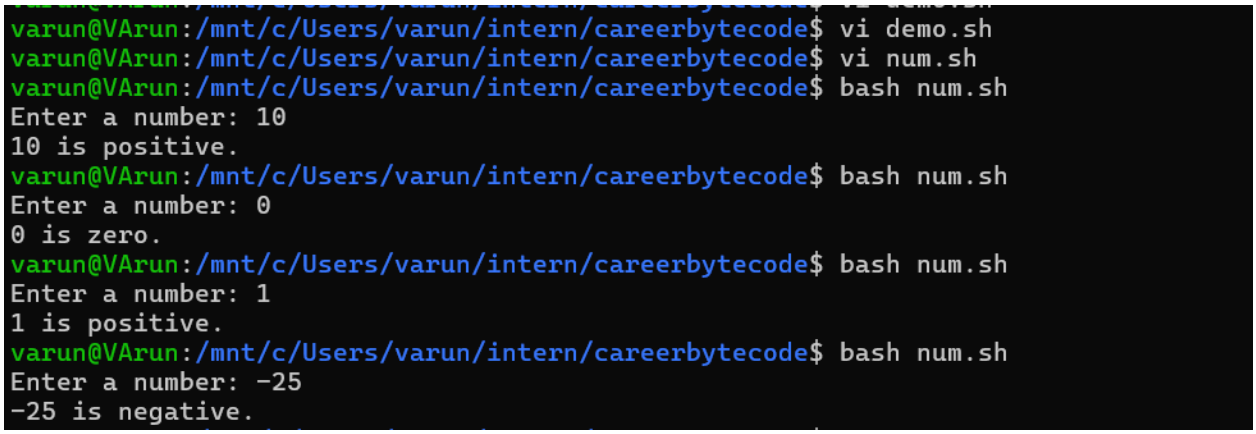
echo "Script name (\$0):        $0"
echo "First argument (\$1):     $1"
echo "Second argument (\$2):    $2"
echo "Number of args (\$#):     $#"
echo "All args with \$@:        $@"
echo "All args with \$*:        $*"
echo "Current PID (\$\$):       $$"

# Run a command, then check its exit status
ls /tmp > /dev/null 2>&1
echo "Exit status of ls (\$?):  $?"

ls /nonexistent_dir > /dev/null 2>&1
echo "Exit status after fail:  $?"
```

```
varun@VArun:/mnt/c/Users/varun/intern/careerbytecode$ vi demo.sh
varun@VArun:/mnt/c/Users/varun/intern/careerbytecode$ bash demo.sh hello world
Script name ($0):        demo.sh
First argument ($1):     hello
Second argument ($2):    world
Number of args ($#):     2
All args with $@:        hello world
All args with $*:        hello world
Current PID ($$):        5010
Exit status of ls ($?):  0
Exit status after fail:  2
```

18. Write a shell script to determine whether a number is positive, negative, or zero using if, elif and else ?




```
#!/bin/bash

# Prompt for a number
read -p "Enter a number: " num

# Check the sign using if / elif / else
if [ "$num" -gt 0 ]; then
   echo "$num is positive."
elif [ "$num" -lt 0 ]; then
   echo "$num is negative."
else
   echo "$num is zero."
fi
```

19. Research three open-source devops tools that work well with linux and git. Explain their purpose and one real-world use case for each.

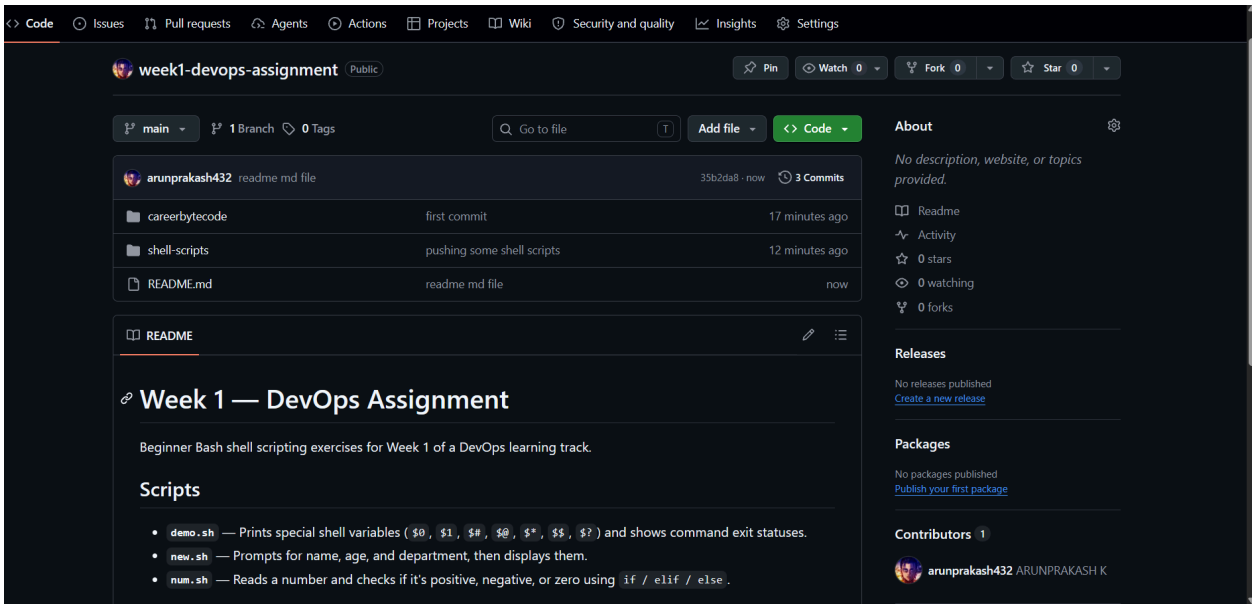
- **a. Ansible**
  - i. It describe server state in yaml playbooks, apply over ssh. Playbooks live in git for version-controlled infrastructure.
- **b. Argocd**
  - i. Git repo is the source of truth for your cluster. Argocd will automatically deploy after each updates in the git history
- **c. Prometheus**
  - i. Scrapes metrics into a time-series DB, queries , promql, scrape configs from git and it will stay alert

20. Create a github repository named week1-devops-assignment containing:

a. **A README file**
b. **Linux command practice**
c. **One bash shell script**
d. **At least five git commits**
e. **Screenshots of executed commands**

```
varun@VArun:/mnt/c/Users/varun/intern$ git config --global user.name "arunprakash432"
varun@VArun:/mnt/c/Users/varun/intern$ git config --global user.email "prakasharun484@gmail.com"
varun@VArun:/mnt/c/Users/varun/intern$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /mnt/c/Users/varun/intern/.git/
varun@VArun:/mnt/c/Users/varun/intern$ cd careerbytecode/
varun@VArun:/mnt/c/Users/varun/intern/careerbytecode$ ls
1  2  3  33  code  code123  demo.sh  new.sh  newdirectory  newfile.txt  num.sh
varun@VArun:/mnt/c/Users/varun/intern/careerbytecode$ cd newdirectory/
varun@VArun:/mnt/c/Users/varun/intern/careerbytecode/newdirectory$ cd ..
varun@VArun:/mnt/c/Users/varun/intern/careerbytecode$ cd ..
varun@VArun:/mnt/c/Users/varun/intern$ git add .
varun@VArun:/mnt/c/Users/varun/intern$ git commit -m "first commit"
[master (root-commit) 68b28b6] first commit
 10 files changed, 61 insertions(+)
 create mode 100644 careerbytecode/1
 create mode 100644 careerbytecode/2
 create mode 100644 careerbytecode/3
 create mode 100644 careerbytecode/33
 create mode 100644 careerbytecode/code
 create mode 100644 careerbytecode/code123
 create mode 100644 careerbytecode/demo.sh
 create mode 100644 careerbytecode/new.sh
 create mode 100644 careerbytecode/newfile.txt
 create mode 100644 careerbytecode/num.sh
varun@VArun:/mnt/c/Users/varun/intern$ git branch -M main
varun@VArun:/mnt/c/Users/varun/intern$ git remote add origin https://github.com/arunprakash432/week1-devops-assignment.git
varun@VArun:/mnt/c/Users/varun/intern$ git push -u origin main
Username for 'https://github.com': arunprakash432
Password for 'https://arunprakash432@github.com':
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
```

```
varun@VArun:/mnt/c/Users/varun/intern$ git push -u origin main
Username for 'https://github.com': arunprakash432
Password for 'https://arunprakash432@github.com':
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (8/8), 997 bytes | 22.00 KiB/s, done.
Total 8 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/arunprakash432/week1-devops-assignment.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
varun@VArun:/mnt/c/Users/varun/intern$ ls
careerbytecode
varun@VArun:/mnt/c/Users/varun/intern$ mkdir shell-scripts
varun@VArun:/mnt/c/Users/varun/intern$ cd ..
varun@VArun:/mnt/c/Users/varun$ cd intern/
varun@VArun:/mnt/c/Users/varun/intern$ cd care
-bash: cd: care: No such file or directory
varun@VArun:/mnt/c/Users/varun/intern$ cd careerbytecode/
varun@VArun:/mnt/c/Users/varun/intern/careerbytecode$ ls
1  2  3  33  code  code123  demo.sh  new.sh  newdirectory  newfile.txt  num.sh
varun@VArun:/mnt/c/Users/varun/intern/careerbytecode$ cp demo.sh new.sh num.sh /mnt/c/Users/varun/intern/shell-scripts/
varun@VArun:/mnt/c/Users/varun/intern/careerbytecode$ cd ..
varun@VArun:/mnt/c/Users/varun/intern$ git add .
varun@VArun:/mnt/c/Users/varun/intern$ git commit -m "pushing some shell scripts"
[main 95046b4] pushing some shell scripts
 3 files changed, 43 insertions(+)
 create mode 100644 shell-scripts/demo.sh
 create mode 100644 shell-scripts/new.sh
 create mode 100644 shell-scripts/num.sh
varun@VArun:/mnt/c/Users/varun/intern$ git push -u origin main
Username for 'https://github.com': arunprakash432
Password for 'https://arunprakash432@github.com':
remote: Invalid username or token. Password authentication is not supported for Git operations.
fatal: Authentication failed for 'https://github.com/arunprakash432/week1-devops-assignment.git/'
varun@VArun:/mnt/c/Users/varun/intern$ git push -u origin main
Username for 'https://github.com': arunprakash432
Password for 'https://arunprakash432@github.com':
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
```


Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
week1-devops-assignment
Public
Pin
Watch 0
Fork 0
Star 0
main
1 Branch
0 Tags
Go to file
T
Add file
Code
arunprakash432 readme md file
35b2da8 · now
3 Commits
careerbytecode first commit 17 minutes ago
shell-scripts pushing some shell scripts 12 minutes ago
README.md readme md file now
README
Week 1 — DevOps Assignment
Beginner Bash shell scripting exercises for Week 1 of a DevOps learning track.
Scripts
demo.sh — Prints special shell variables ($0 , $1 , $# , $@ , $* , $$, $?) and shows command exit statuses.
new.sh — Prompts for name, age, and department, then displays them.
num.sh — Reads a number and checks if it's positive, negative, or zero using if / elif / else .
About
No description, website, or topics provided.
Readme
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package
Contributors 1
arunprakash432 ARUNPRAKASH K